Recycle Rush

Report and System Documentation

Recycle Rush & Recycle Dash

Abraham Oreem
29 /08/2025

Table of Contents

1. Introduction

The project is a Unity- based game aimed at promoting recycling habits and keeping the environment clean.

1.1. Overview

The game combines two popular game genres: sorting and endless runner. It implements the drag and drop gameplay mechanic for sorting and the player movement with item collection for the endless runner game. The game loop starts from the runner scene where the player runs and collects four types of items: Garbage, Electronic, Compost and Recyclables. After collecting a certain number of items (default 40 items), the player moves to the second scene, where the player has to sort the collected items into their respective bins. Once all items are sorted, the player returns to the runner scene, and the loop continues. In addition, there is a separate standalone game built exclusively for the sorting mechanic.

1.2. Purpose of project

The purpose of this project is to raise awareness about the importance of recycling and environmental cleanliness in a playful and engaging way. The game aims to promote recycling habits and encourage maintaining a clean environment. Additionally, it educates players on how to correctly sort different types of waste into their respective bins. The game combines education and entertainment, teaching players about recycling while providing engaging and enjoyable gameplay.

2. Concept and Gameplay

2.1. Core Idea

The core idea of the Recycle Rush game revolves around a recycling factory truck that releases items onto the road. The game starts with a loading screen and after the loading screen the user is prompted to select character and enter name.

Once the player starts the game the MainGamePlay scene is loaded. In that scene the player controls a character who runs along the road, avoiding obstacles while collecting these items. After collecting a certain number of items, the recycling factory is respawned ahead of the player, triggering an automatic transition to the sorting scene.

Figure 1: Loading and Character Select

In the sorting scene, the collected items are displayed on shelves, and a queue of bins is available for the player to sort the items using a drag-and-drop mechanic. Once all items are correctly sorted, the player returns to the same position in the runner scene and continues collecting items.

This creates an endless runner loop with checkpoints, where players collect items, sort them at the recycling factory, and earn points for correct sorting. By combining fast-paced running with interactive sorting, the game effectively blends entertainment with education on recycling and waste management.

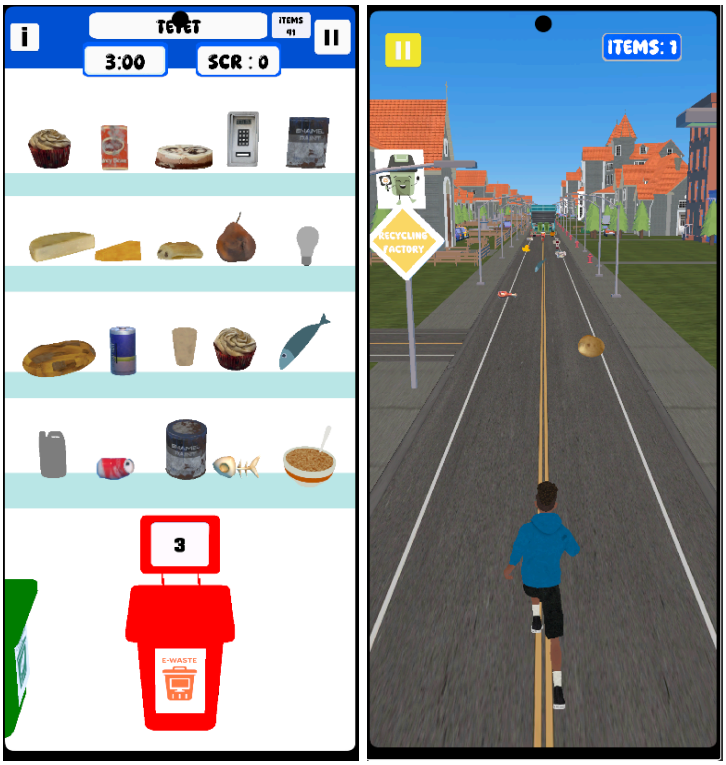


Figure 2: “SortingPhase” and “MainGameplay” scene

2.2. GamePlay

Drag-and-Drop Sorting

- Players pick up collected items from shelves and drag them into the correct bins.
- Items must be sorted according to their type: Garbage, Electronics, Compost, or Recyclables.
- Correct placement rewards points, while incorrect placement deduct points there is an info page to guide the player.
- This mechanic teaches proper recycling habits in an interactive and engaging way.

Runner Controls

- **Lane Changing:** Players swipe left or right to move the character between lanes, avoiding obstacles and collecting items.
- **Jumping:** Swiping up allows the character to jump over obstacles.
- **Sliding:** Swiping down lets the character slide under low-hanging obstacles.

- The player continuously runs forward, collecting items while reacting to obstacles in real-time.

Game Loop & Checkpoints

- Players collect items during the runner phase and reach checkpoints after collecting a certain number of items.
- At checkpoints, the player is transported to the sorting scene to sort the collected items using the drag-and-drop mechanic.
- Once all items are sorted, the player returns to the runner phase at the same position, continuing the endless loop.

Standalone Sorting Game

- A separate game exists solely for practicing the sorting waste.
- Players can focus exclusively on correctly categorizing items without the runner gameplay.

Figure 3: Recycle Rush (Standalone Sorting Game)

2.3. GamePlay Loop Diagram

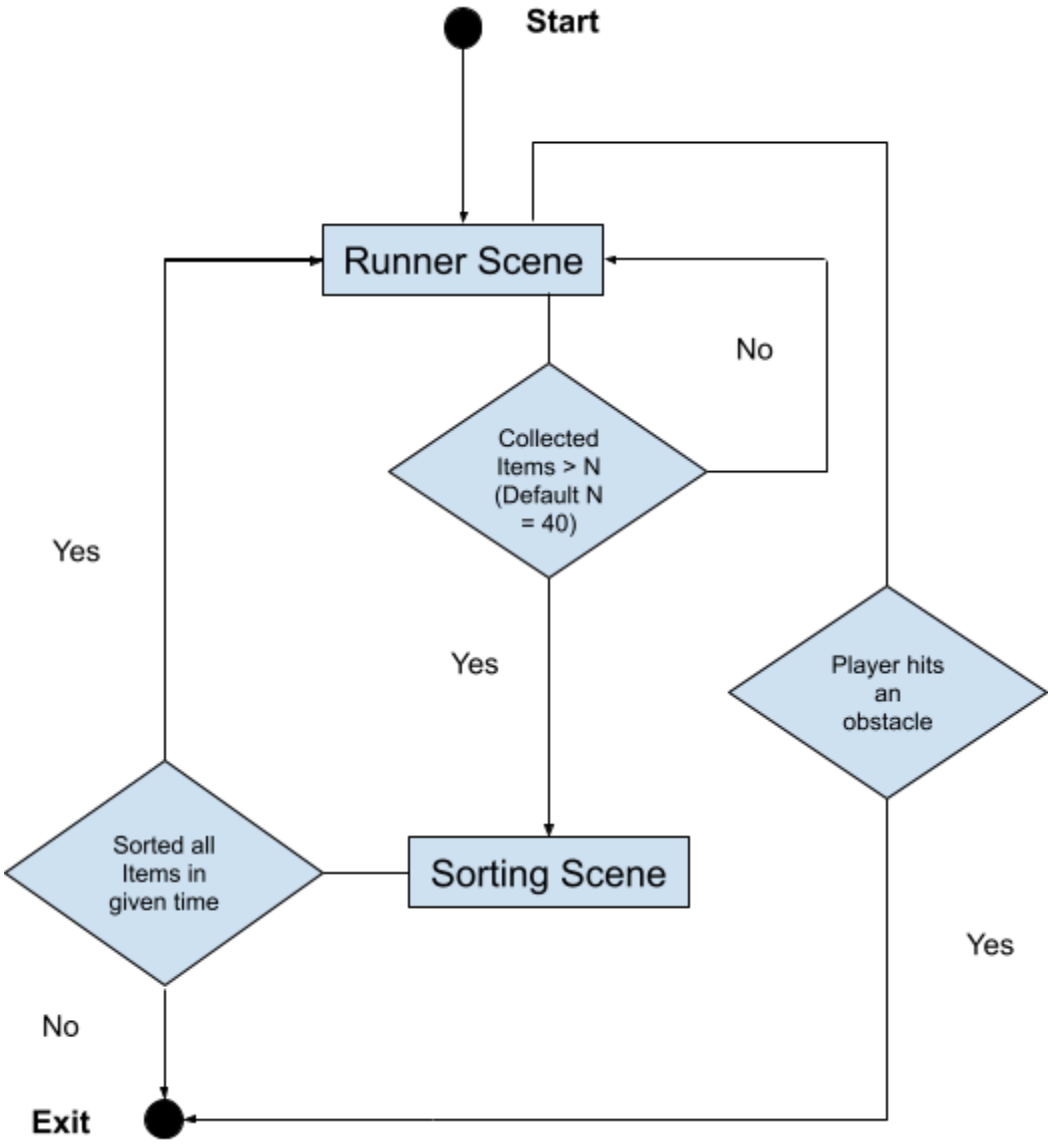


Figure 4: Gameplay loop diagram

3. System Architecture

The architecture of the game has multiple layers. The layers are as follows.

- Game Management Layer
- Player Interaction Layer
- Interactive Objects Layer
- UI & Feedback Layer
- Data Layer

3.1. Game Management Layer

The game management layer controls the basic flow of the game. There are two major scripts in the game management layer. Those two are “RunnerGameManager.cs” and the “GameManager.cs”. RunnerGameManager.cs manages all the game flow for the endless runner part of the game and the GameManager.cs manages all the game flow for the sorting part of the game. Both of these scripts are implemented as singletons and act as central hubs, holding references to all major game components. This design allows other scripts to easily access and invoke their functions through the singleton instance of the GameManager or the RunnerGameManager, streamlining communication across the system.

3.2. Player Interaction Layer

The player interaction layer deals with how the player interacts with the layer. This layer includes scripts like InputManager.cs, PlayerController.cs and the TrashItem.cs. The Input manager detects swipe gestures on a touchscreen and translates them into directional events. It tracks the start and end positions of a touch, compares them against a threshold, and determines the swipe direction. Depending on the detected swipe, it invokes the corresponding event (up, down, left, or right) for other scripts to use. The PlayerController script controls the player by subscribing to swipe events from InputManager, making the character jump, slide, or switch lanes. It also uses DOTween (a Unity plugin) to smoothly handle animations, camera transitions, and lane movements. Almost all the tweening in the game is handled using DOTween. For the sorting part the TrashItem script lets players drag and drop trash items into matching bins, updating scores and bin capacities. It handles mouse input via OnMouseDown, OnMouseDrag, and OnMouseUp, converting screen positions to world positions for smooth movement. DG.Tweening is used for animations like moving, scaling, and pinch effects when items are placed or returned.

3.3. Interactive Objects Layer

This layer deals with all other interactive objects in the game. It includes scripts such as TrashBinManager, GarbageItemManager, ObjectCollider, ItemSpawner and the ObjectPooling script. The TrashBin Manager manages the queue of trash bins in the sorting scene of the game. It also instantiates all the trash bins and handles their queue movement. GarbageItemManager manages the instantiation of TrashItems in the sorting scene of the game. It also manages the dropping of new items into the scene when there are free spaces available on the shelves. An

ObjectCollider script is attached to every object with a collider that can interact with the player. It contains trigger events for each situation. The script utilizes tags of the gameobjects so it is very important that the gameobjects have the right tags assigned in the inspector. The runner game uses object pooling to create an endless world. Environment patches are recycled by moving them in front of the player once left behind, optimizing performance. This avoids constantly instantiating and destroying objects, keeping the game smooth. All of this is handled by the ObjectPooling script. At last there is the ItemSpawner script which spawns the pickup items on the world patches. So that the player can pick those items.

3.4. UI & Feedback Layer

The UI & Feedback layer has two main scripts: the UIManager and the UIPopup script. The UIManager script manages the game's UI, updating item counts, levels, sliders, and showing win/lose panels, while handling scene loading, pausing, and sound toggles. It also saves player preferences like slider values and displays random fun facts on wins.

The UIPopup script controls pop-in and hides animations for UI elements using DG.Tweening, smoothly scaling UI objects in and out when enabled or hidden. It ensures tweens don't conflict by killing any existing animations before starting new ones. Additionally the UI-effect plugin by mob-sakai is used to improve the visuals. Specifically the appearance of buttons and different panels such as the top score panel.

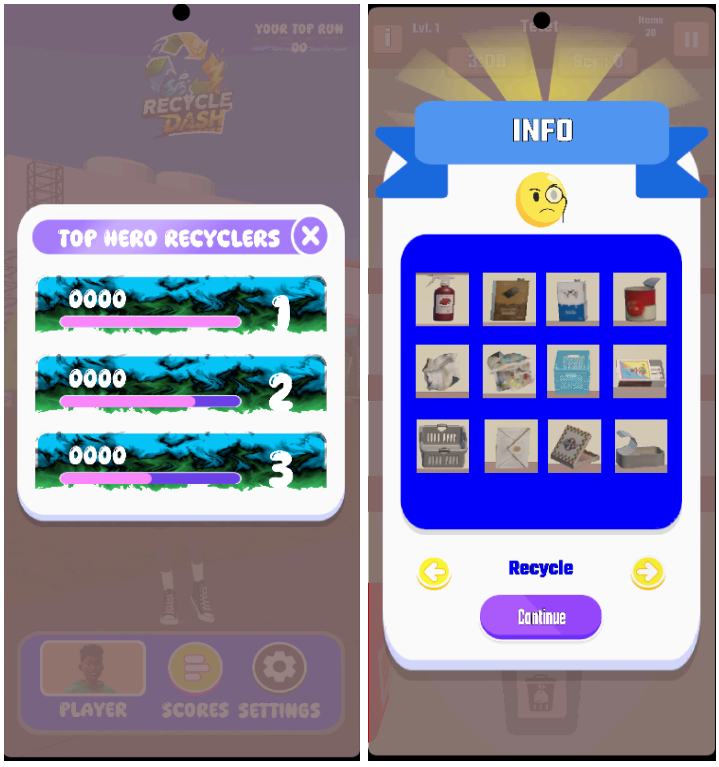


Figure 5: UI Samples

3.5. Data Layer

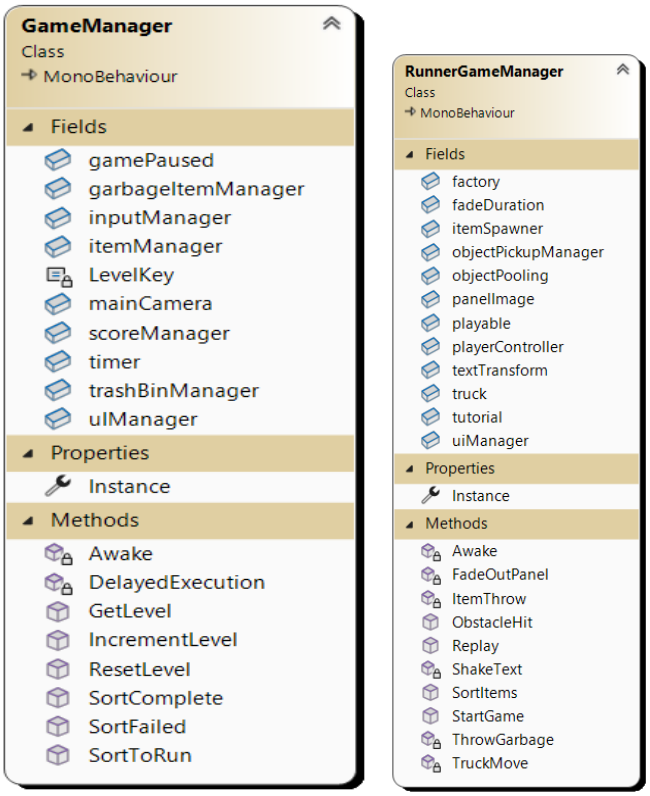
The Data Layer manages all player and game data both locally and on the server, ensuring persistence and synchronization across sessions. The game uses PlayerPrefs for local storage and PlayFab for cloud storage, allowing scores and player information to be stored securely and accessed globally.

The PlayFabLogin script handles player authentication, logging in with a device's unique ID and creating a PlayFab account if needed. On successful login, it triggers leaderboard updates by fetching top players depending on the scene, top three for the runner game and a single top player for the sorting game.

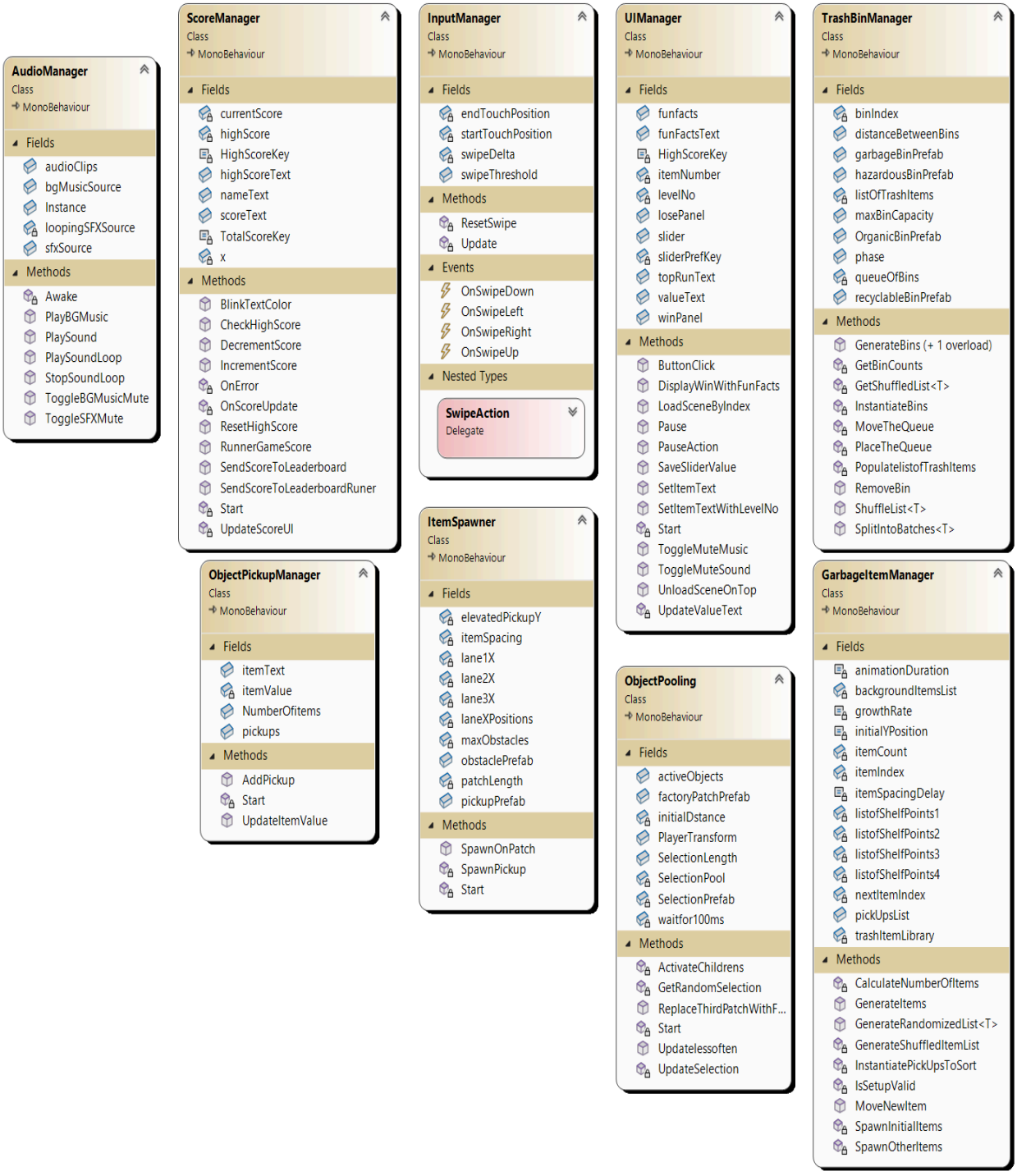
The ScoreManager script manages scores, high scores, and leaderboard updates. It updates scores locally via PlayerPrefs and sends them to the PlayFab leaderboard using UpdatePlayerStatisticsRequest. The runner game tracks the top three scores, while the sorting game shows the single highest score. This layered design ensures seamless data flow between local storage, server, and UI, keeping gameplay consistent and responsive.

4. Code Structure and Components

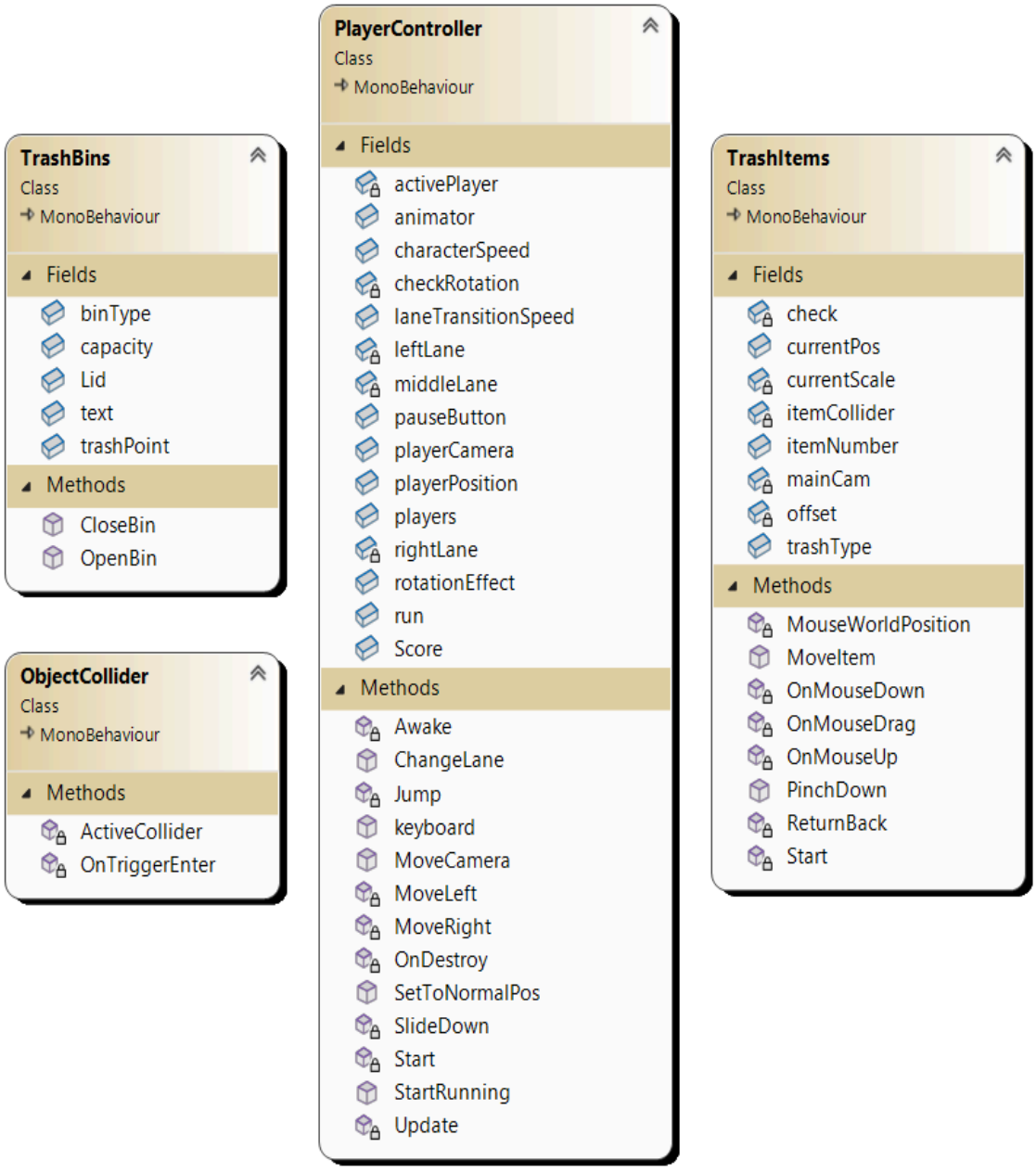
4.1. Singletons:



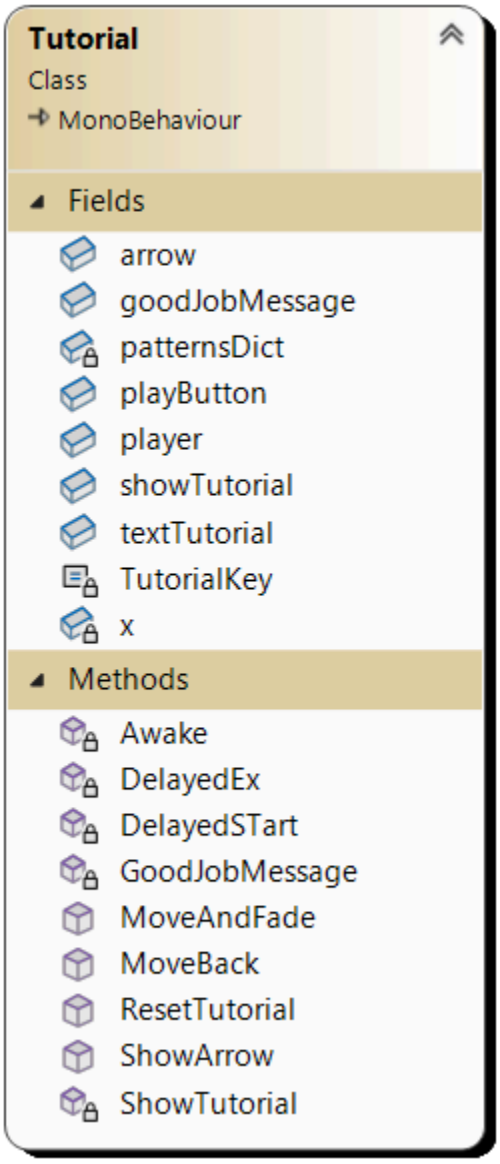
4.2. Game Managers:

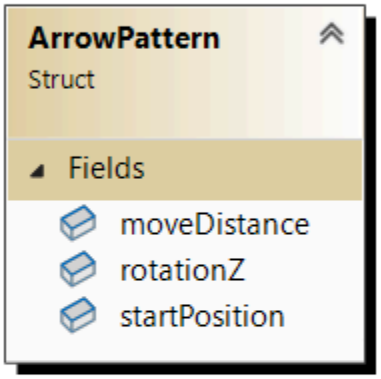


4.3. Core Scripts

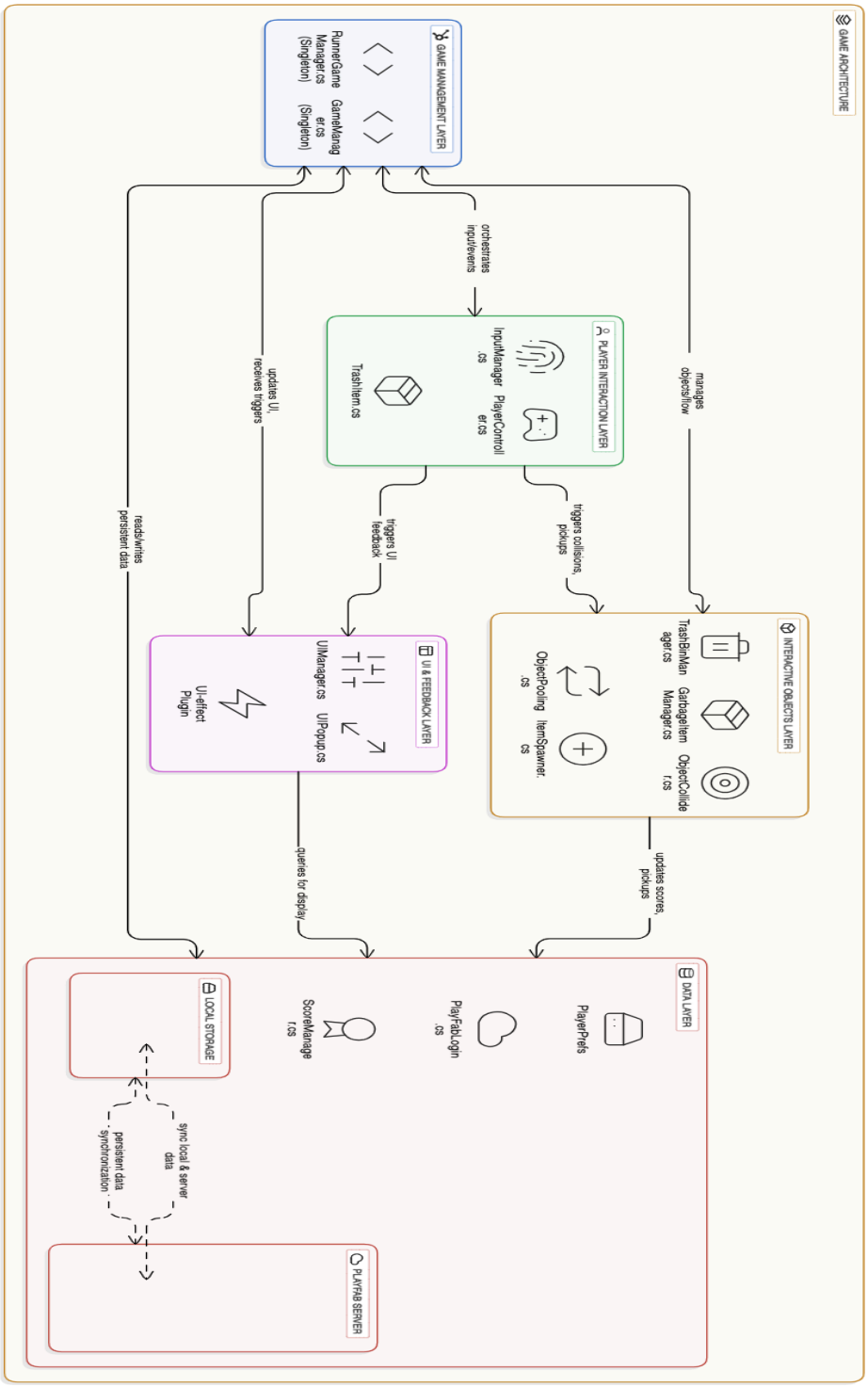


4.4. Tutorial


Tutorial
Class
MonoBehaviour
Fields
arrow
goodJobMessage
patternsDict
playButton
player
showTutorial
textTutorial
TutorialKey
x
Methods
Awake
DelayedEx
DelayedSTart
GoodJobMessage
MoveAndFade
MoveBack
ResetTutorial
ShowArrow
ShowTutorial


ArrowPattern
Struct
Fields
moveDistance
rotationZ
startPosition

5. Game Architecture Diagram



6. Project Setup

6.1. Setting up the Unity Project from GitHub

The project is hosted on GitHub at: https://github.com/PASCL-Lab/Recycling-Games

The Unity version for the project is 2022.3.62f1. To set up the project locally, follow these steps:

1. Clone the repository from the link above or download the project as a zip file from the github application and extract the project.
2. Open the project in Unity. Ensure the Unity version matches the project's version. Unity will automatically upgrade/downgrade if needed.
3. Unity will automatically install all the libraries and dependencies. But if there are any missing, go to the packages / manifest.json file in the project. And make sure every listed package is installed.
4. Navigate to the Assets/GameData/Scenes folder and open the LoadingScreen scene to start the game.
5. For the standalone sorting game go to Assets/GameData/Scenes and open the Menu scene to start the game.

6.2. PlayFab Integration

The game uses PlayFab to store player scores and manage leaderboards. To set up PlayFab:

6. Create a PlayFab Account
    a. Go to PlayFab and create an account if you don't have one.
7. Create a New Title/Game
    a. In PlayFab, create a new title for your project. It is important to create two titles as there are two games in the project "Recycle Dash" and "Recycle Rush". And there should be different leaderboards for both games.
8. Create a Legacy Leaderboard Table
    a. Create a legacy leaderboard table named "High_Score" to store player scores. It is important that the name of the table is "High_Score" for both the game titles on PlayFab. The name can be changed by going to the ScoreManager script and changing the "StatisticName" attribute of every UpdatePlayerStatisticsRequest in the script.

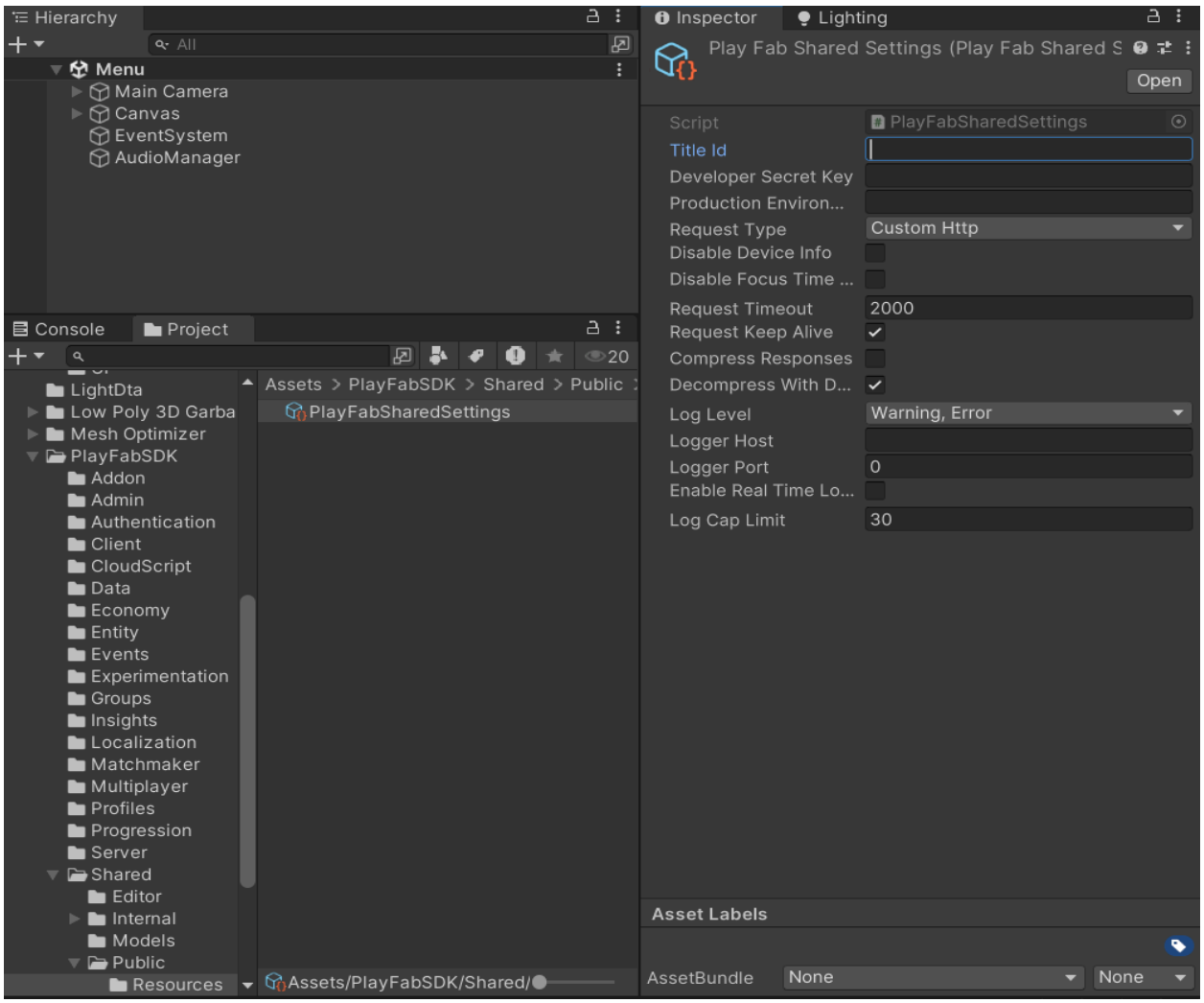
9. Configure PlayFab in Unity

In Unity, navigate to:

Assets → PlayFabSDK → Shared → Public → Resources

a. Click on PlayFabSharedSettings. As shown in the figure below.

b. Enter the Title ID of the PlayFab game in the Title Id field. For building “Runner Dash” enter the title id for the runner dash game and for building “Recycle Runner” enter the title id for the Recycle Runner game.



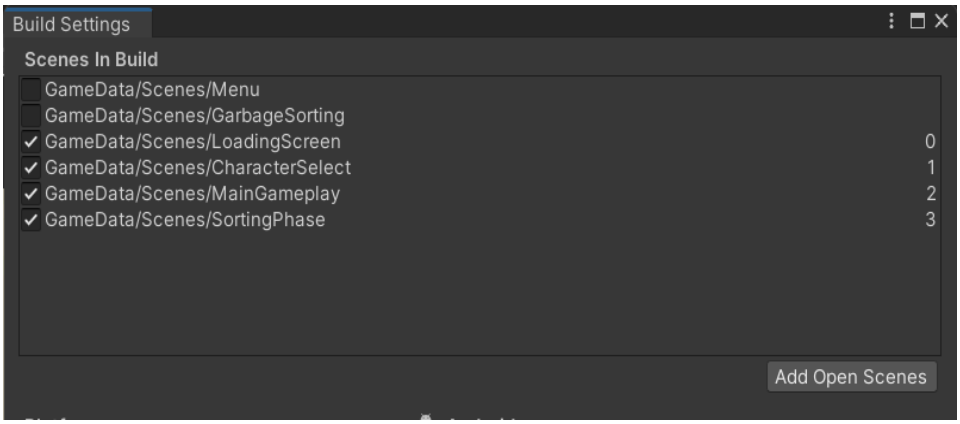
10. Verify Integration

a. Play the game in Unity and ensure that scores are saved to and retrieved from PlayFab correctly.

6.3. Android Builds

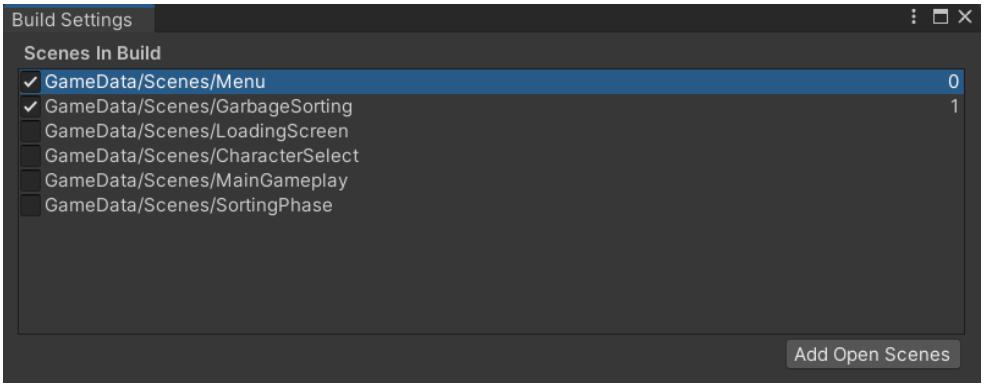
6.3.1. Building the Recycle Dash Game

- Before building the game make sure that the title id is set correctly for the Recycle Dash game in the PlayFabSharedSettings. To set the title id follow the steps given in section 5.2.
- The following figure shows the scenes that are included in the game build for Recycle Dash. They must appear in the same sequence as listed.



6.3.2. Building the Recycle Rush Game

- Similarly before building the game make sure that the title id is set correctly for the Recycle Rush game in the PlayFabSharedSettings. To set the title id follow the steps given in section 5.2.
- The following figure shows the scenes that are included in the game build for Recycle Rush. They must appear in the same sequence as listed.



7. Conclusion & Outcomes

In this project, we successfully implemented an endless runner style game and a drag a drop sorting game in Unity, incorporating core gameplay mechanics such as swipe-based player controls, drag and drop items, collision detection, and dynamic score management. Integration with PlayFab allowed us to manage player authentication and maintain a centralized leaderboard, making the game scalable and engaging for multiple users. DOTween was utilized for smooth animations and UI feedback, enhancing the overall user experience. By structuring the project with modular scripts such as InputManager, PlayFabLogin, ScoreManager, and ObjectCollider, the system is both extensible and maintainable for future updates. Furthermore, the inclusion of the sorting phase makes the game not only entertaining but also educational, as it encourages players to learn trash sorting and promotes awareness about maintaining a clean environment.

Outcomes:

- Implemented swipe-based input controls for intuitive player movement on mobile devices.
- Integrated collision-based events, enabling trash pickups, obstacle interactions, and triggering the factory/sorting phase.
- Designed a sorting mechanic where players organize collected trash, adding depth to the gameplay loop.
- Developed score tracking with both local persistence (PlayerPrefs) and cloud synchronization via PlayFab.
- Established a leaderboard system on PlayFab using the legacy table *High_Score*, allowing global competition.
- Ensured smooth transitions between phases (running → factory/sorting → scoring).
- Enhanced UI feedback and animations with DOTween for real-time score updates and interaction clarity.
- Verified Android builds, maintaining correct scene order and ensuring stable performance across devices.

The final build was successfully deployed for Android, verifying smooth integration of PlayFab services and demonstrating stable gameplay performance across devices.